MAZE-X

Interactive Maze Generation, Pathfinding
and DSA Visualization Platform

Complete Project Overview

Language	C++17 with the STL
Graphics (later phase)	SFML
Environment	Windows, VS Code, g++ / MinGW-w64
Repository	github.com/Niraj120307/MazeX
Local folder	D:\MazeX
Current status	V0.1 Foundation complete (console maze model + tests)

1. Project summary

MAZE-X is an educational algorithm laboratory. It generates and edits mazes, runs graph-search algorithms on them step by step, and compares the resulting paths and measured performance through an interactive SFML interface. It is a DSA course project, not a game and not an AI model.

The problem it solves: pathfinding is usually taught as pseudocode or console output, so students cannot see how the choice of data structure changes the search. MAZE-X turns the maze into a visible grid graph so the search process, the final path and the metrics can all be inspected.

Objectives

- Represent a maze as a grid graph with walls, a start, a destination and optional weighted terrain.
- Implement DFS, BFS, Dijkstra and A* from scratch using STL containers (no pathfinding library).
- Animate the search so explored cells and the final route are easy to tell apart.
- Show the algorithm's main data structure live: stack, queue or priority queue.
- Measure nodes explored, path length/cost and execution time on the same maze.
- Provide maze editing, generation, reset and save/load.
- Deliver a polished, explainable demo suitable for a viva.

The core idea

Every walkable cell is a vertex. Every legal move between two neighbouring walkable cells is an edge. All the search algorithms share one loop. The only thing that changes is the data structure holding the cells waiting to be explored.

Algorithm	Data structure	Behaviour	Guarantee	Complexity
DFS	Stack (or recursion)	Goes deep first, then backtracks	Finds a path if one exists; not necessarily the shortest	O(V + E)
BFS	Queue (FIFO)	Expands outward in rings from the start	Shortest path by number of moves (unweighted)	O(V + E)
Dijkstra	Min-priority queue by cost so far	Always expands the cheapest known cell	Minimum total cost, non-negative weights	O((V + E) log V)
A*	Min-priority queue by f = g + h	Like Dijkstra but pulled toward the goal	Optimal with an admissible heuristic	Depends on heuristic; worst case can be exponential

2. Requirements and scope

Functional requirements

ID	Requirement	Priority
FR-01	Display a grid maze with start, end, walls and walkable cells	Must
FR-02	Draw and erase walls with the mouse	Must
FR-03	Run DFS and BFS and reconstruct the found path	Must
FR-04	Animate visited/explored cells and the final path	Must
FR-05	Play, pause, step, reset and speed controls	Must
FR-06	Show nodes explored, path length/cost and elapsed time	Must
FR-07	Generate a maze with recursive backtracking	Should
FR-08	Weighted terrain and Dijkstra's algorithm	Should
FR-09	A* with a documented heuristic	Should
FR-10	Live stack / queue / priority-queue display	Should
FR-11	Compare multiple algorithms on the same maze	Should
FR-12	Save and load maze layouts	Could
FR-13	Batch benchmarks on multiple grid sizes	Could
FR-14	Race mode, challenge mode, themes, extra generators	Stretch

Non-functional requirements

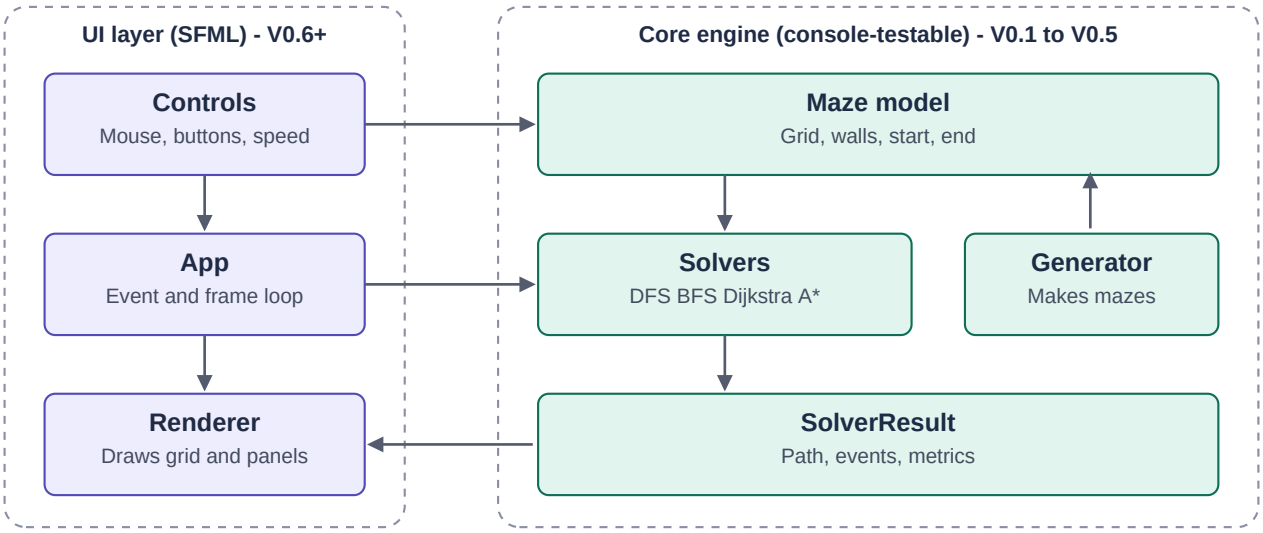
- **Correctness:** a returned path starts at S, ends at E, avoids walls and uses only legal neighbouring moves.
- **Responsiveness:** the UI stays interactive during animation; use a frame/event loop, never blocking sleeps.
- **Maintainability:** keep maze data, solvers, rendering and metrics in separate modules.
- **Usability:** visible legend, clear status messages, and a reset path after every run.
- **Portability:** standard C++ and documented SFML setup; no machine-specific paths.
- **Honest measurement:** label timing as an observed run, repeat benchmarks, never claim one algorithm is always fastest.

Scope recommendation

Must finish: grid model, manual editing, DFS, BFS, parent-based path reconstruction, animated search, play/pause/step/reset and truthful metrics. **Strong final version:** add recursive-backtracking generation, Dijkstra, A*, weighted terrain, live data-structure panels and comparison. **Stretch:** race mode, benchmark CSV, multiple generators, challenge mode, themes. A correct, well-explained project with a clean visualizer scores better than a long feature list with unreliable algorithms.

3. Architecture and data flow

The core engine is independent of the GUI. A solver receives a snapshot (copy) of the maze and returns results; the renderer only displays those results. This keeps algorithms testable in the console and prevents SFML code from leaking into every solver.



How a run works

- The user edits walls through **Controls**; the **App** loop applies edits to the **Maze model**. The **Generator** can also write into the maze.
- On Solve, the App hands a copy of the maze to a solver, so every algorithm gets identical input.
- The solver returns one common **SolverResult**: path, visited order, nodes visited, cost and time.
- The **Renderer** draws that result and never runs algorithm logic, so animation speed cannot change correctness or timing.

Planned common result structure

```
struct SolverResult {
    bool               found;         // was a path found?
    std::vector<Point> path;          // start -> end, in order
    std::vector<Point> visitedOrder;  // cells in the order explored
    int                nodesVisited;
    double             pathCost;      // moves for unweighted, total cost for weighted
    double             elapsedMs;     // solver time only (no animation/rendering)
};
```

Key design rules

- Permanent maze data (walls, terrain) is never overwritten by temporary search state. Visited, frontier and path live in a separate overlay.
- A fixed neighbour order (Up, Right, Down, Left) is used everywhere so runs are repeatable and comparable.
- Path length means the number of moves (edges), which is the number of path cells minus one.

4. Folder structure

```
MazeX/
|-- .vscode/
|   `-- settings.json
|-- include/                declarations (the "what")
|   |-- Point.h             V0.1
|   |-- Cell.h              V0.1
|   |-- Maze.h              V0.1
|   |-- DFS.h  BFS.h        V0.2
|   |-- SolverResult.h      V0.3
|   |-- Dijkstra.h AStar.h  V0.4
|   `-- MazeGenerator.h     V0.5
|-- src/                    implementations (the "how")
|   |-- main.cpp            V0.1
|   |-- Maze.cpp            V0.1
|   |-- DFS.cpp  BFS.cpp    V0.2
|   |-- Dijkstra.cpp AStar.cpp   V0.4
|   |-- MazeGenerator.cpp   V0.5
|   `-- ui/                 V0.6+ (SFML only)
|       |-- App.cpp  Renderer.cpp  Controls.cpp
|-- tests/
|   `-- test_maze.cpp       V0.1
|-- benchmarks/
|-- assets/
|-- .gitignore
|-- README.md
`-- CMakeLists.txt          optional
```

A header (.h) declares a class or function interface; a source file (.cpp) contains its implementation. Other files include the header to use the class without seeing its internals. Include guards or `#pragma once` stop a header being included twice.

5. Technology stack

Tool	Purpose	Required?
C++17	Core application, OOP, solvers, timing	Yes
STL (vector, queue, stack, priority_queue)	Grid storage and algorithm data structures	Yes
VS Code + C/C++ extension	Editor, IntelliSense, debugging	Yes
g++ / MinGW-w64	Compile on Windows (must be on PATH)	Yes
SFML	Window, graphics, text, input, rendering	GUI stage only
Git + GitHub	Version control and milestone history	Strongly recommended
CMake	Repeatable build configuration	Optional
CSV / text file I/O	Save mazes, export benchmarks	Later

SFML note: use a version that matches your compiler and architecture, and test a tiny SFML window early in the GUI phase. Keep the algorithm engine free of SFML so it can always be tested in the console.

6. Development roadmap

Phase	Name	What you build	Done when
V0.1	Foundation	Point, CellType, Maze, console display, start/end/walls	Maze can be created, edited and printed reliably (COMPLETE)
V0.2	Core DSA	DFS, BFS, visited array, parent array, path reconstruction, no-path handling	Both pass hand-made test mazes
V0.3	Results	SolverResult: path, visited order, nodes, cost, time	All solvers return comparable results
V0.4	Advanced solvers	Terrain costs, Dijkstra, A* with Manhattan heuristic	A* cost equals Dijkstra cost on tests
V0.5	Generation	Random obstacles, recursive-backtracking generator	Generated mazes are valid and solvable
V0.6	SFML UI	Window, grid drawing, mouse editing, legend	Create, edit and solve graphically
V0.7	Animation	Event playback, play/pause/step/speed, overlays	Search visible; UI never freezes
V0.8	DSA panels	Live stack / queue / priority-queue view	Panel shows real solver state
V0.9	Comparison	Run all algorithms on the same snapshot; results table	Metrics use identical input
V1.0	Final polish	Save/load, CSV export, README, screenshots, demo script	Repeatable and presentation-ready

Workflow rules

- Do one milestone at a time and compile after each meaningful change.
- Commit each working milestone to Git instead of waiting until the end.
- Keep a known-good small maze for regression testing.
- Do not start SFML until the console solvers and path reconstruction are trustworthy.
- If time runs short, keep DFS/BFS, visualization, metrics and one generator; defer extra modes.

7. V0.1 Foundation: what is built

V0.1 delivers the maze model and its tests. It compiles with no warnings under `-Wall -Wextra -pedantic`, and all 46 test checks pass.

File	Responsibility
include/Point.h	(row, col) coordinate with ==, !=, <, +, printing and a hash for unordered containers
include/Cell.h	CellType { Empty, Wall, Start, End } and the display character for each
include/Maze.h	Maze interface and the fixed neighbour order kDirections
src/Maze.cpp	Grid stored as a flat vector<bool>; index = row * cols + col; all edits bounds-checked
src/main.cpp	Console demo: draw walls, try invalid edits, print neighbours
tests/test_maze.cpp	46 checks: construction, bounds, walls, start/end rules, neighbours, display

Maze public interface

```
Maze(int rows, int cols);                // >= 2 x 2, else std::invalid_argument
int   rows() const;   int cols() const;
Point start() const;  Point end() const;  int wallCount() const;

bool inBounds(Point p) const;
CellType typeAt(Point p) const;          // throws std::out_of_range if outside
bool isWall(Point p) const;              // false when out of bounds
bool isWalkable(Point p) const;          // in bounds and not a wall
std::vector<Point> neighbors(Point p) const;   // order: Up, Right, Down, Left

bool setWall(Point p);     bool removeWall(Point p);   bool toggleWall(Point p);
bool setStart(Point p);    bool setEnd(Point p);       void clearWalls();
std::string toString() const;            // '#' wall  '.' empty  'S' start  'E' end
```

Design decisions

Decision	Choice
Coordinates	Point{row, col}; (0,0) is top-left
Movement	4-direction, fixed order Up, Right, Down, Left
Storage	Maze stores only walls; Start and End are separate Points, so a cell can never be both a wall and a marker
Default markers	Start = top-left, End = bottom-right
Minimum size	2 x 2 so Start differs from End
Start == End	Not allowed (setStart / setEnd reject it)
Invalid edits	Return false; never crash, never read outside the grid
Search state	Not stored in Maze; a separate overlay will be added in V0.7

8. Build, run and test (Windows)

Run these from `D:\MazeX` in the VS Code terminal.

```
g++ --version

:: build and run the demo
g++ -std=c++17 -Wall -Wextra -Iinclude src/main.cpp src/Maze.cpp -o MazeX.exe
.\MazeX.exe

:: build and run the tests (expect: 46/46 checks passed)
g++ -std=c++17 -Wall -Wextra -Iinclude src/Maze.cpp tests/test_maze.cpp -o test_maze.exe
.\test_maze.exe
```

The `-Iinclude` flag tells the compiler where the headers live. Without it you get "Maze.h: No such file".

Troubleshooting: VS Code IntelliSense warning on Maze.h

If the Problems panel shows "inline specifier allowed on function declarations only", VS Code is treating the header as C. This is an editor warning, not a compile error. Create `.vscode/settings.json` with the content below and reload the window.

```
{
    "C_Cpp.default.cppStandard": "c++17",
    "C_Cpp.default.includePath": ["${workspaceFolder}/include"],
    "files.associations": { "*.h": "cpp" }
}
```

Git workflow

```
cd /d/MazeX
git init
git branch -M main
git remote add origin https://github.com/Niraj120307/MazeX.git
git add .
git commit -m "Add maze grid model"
git push -u origin main
```

If `origin` already exists, do not add it again; check with `git remote -v`.

9. Algorithm correctness notes (for V0.2 onward)

- **BFS:** mark a cell as discovered when it is enqueued, not when it is dequeued, to avoid duplicate queue entries.
- **DFS:** a visited check is mandatory; without it the search loops forever on open grids.
- **Parent array:** store, for each cell, the cell it was discovered from. To build the path, walk from the End back to the Start and reverse.
- **Dijkstra / A*:** ignore stale priority-queue entries, or ensure a node is processed with its best known score.
- **A* heuristic:** Manhattan distance is admissible for 4-direction movement. With weighted terrain, scale it by the minimum step cost so it never overestimates.
- **Timing:** tiny mazes give noisy times. Keep animation out of the measurement and repeat runs in benchmark mode.

BFS outline

```
queue.push(start);  discovered[start] = true;  parent[start] = none
while queue not empty:
    cur = queue.pop_front()
    if cur == end: break
    for n in maze.neighbors(cur):        // Up, Right, Down, Left
        if not discovered[n]:
            discovered[n] = true;  parent[n] = cur;  queue.push(n)
if end not discovered: found = false
else: walk parent[] from end back to start, then reverse
```

10. Testing and benchmarking plan

Case	Expected behaviour
Start equals destination	Rejected by design (V0.1); solvers never see it
Direct corridor	Finds the obvious path; cost equals moves for unit weights
Wall forces a detour	Legal detour found; BFS returns the minimum number of moves
Destination fully blocked	found = false, show "No path found", no crash
Start/end on a wall	Edit rejected
Maze border	Neighbour checks never read outside the grid
Repeated solve	Old visited/path overlays cleared before the new run
Weighted alternative	Dijkstra/A* prefer a lower-cost route even with more moves
Generated maze	No invalid coordinates; follows its documented connectivity rules
Resize / close window	UI handles both without hanging

Benchmark rules

- Run all algorithms on an identical copy of the same maze.
- Separate solver time from animation and rendering time.
- Report nodes explored and path length/cost alongside time; time alone is misleading.
- Repeat runs and report a median or average, especially on small grids.
- Never hard-code sample metrics; populate the dashboard from real solver results.
- Document movement rules, neighbour order, terrain costs and whether start/end count as path cells.

11. UI plan

Area	Contents
Top bar	MAZE-X title, subtitle, status (Ready / Searching / Solved / No path)
Left sidebar	Algorithm selector, maze generation, clear/reset, edit mode, terrain tools
Center canvas	Resizable grid: walls, start/end, frontier, visited cells, final path
Right panel	Selected algorithm, data structure, current node, complexity, live metrics
Bottom controls	Play, pause, next step, restart, animation speed
Optional lower panel	Live queue / stack / priority queue and algorithm explanation

Visual language

Visual state	Meaning
Dark solid tile	Wall / blocked cell
Neutral tile	Walkable and not yet explored
Blue / cyan overlay	Frontier or currently exploring
Muted blue overlay	Visited / explored
Bright green / yellow route	Final path
Green marker / red marker	Start / destination
Terrain colour or pattern	Weighted cost, with legend and optional numeric cost

Interaction plan

- Mouse click/drag draws walls; a separate erase tool or right-click removes them.
- Generate creates a new maze and clears old search overlays.
- Pause and step stop at a stable event boundary.
- Reset clears search overlays and keeps the maze. Clear removes the walls. Use distinct labels for the two.
- Speed changes the delay between visualization events, not the solver's benchmark timing.

12. Demo script (4 to 6 minutes)

Step	Show	Explain
1	Open MAZE-X, point out legend and controls	Grid as graph: vertices, edges, cell states
2	Draw a wall, move start/end	UI updates the maze model and validates cells
3	Run DFS with animation and step mode	Stack/recursion, visited cells, why DFS is not shortest
4	Reset, run BFS on the same maze	Queue, level-by-level exploration, shortest in unweighted grid
5	Enable terrain, run Dijkstra / A*	Weighted cost, priority queue, heuristic
6	Open the comparison panel	Nodes explored, path length/cost, time, fair comparison
7	Generate a maze, optionally save/load	Backtracking generation and file persistence

Use a prepared maze so the demo does not depend on random output.

13. Viva preparation

Question	Short answer
Why can a grid maze be modelled as a graph?	Each walkable cell is a vertex and each legal move to an adjacent walkable cell is an edge.
Why does BFS give the shortest path?	The FIFO queue processes cells in order of increasing distance, so the first time it reaches the end uses the fewest moves (unweighted graph).
Why doesn't DFS guarantee the shortest path?	It commits deep along one branch and returns the first path it finds, which may be long.
Why does Dijkstra need non-negative weights?	It finalizes the cheapest cell first. A negative edge could later make an already-finalized cell cheaper, breaking that assumption.
What does f(n) = g(n) + h(n) mean?	g is the cost so far from the start, h is the estimated remaining cost to the goal, and A* expands the lowest f.
Why is Manhattan distance suitable?	With 4-direction moves the fewest possible moves is |dr| + |dc|, so it never overestimates (admissible).
What does the parent array do?	Records which cell each cell was discovered from, so the path is rebuilt by walking back from the end to the start.
Time and space complexity?	DFS and BFS are O(V + E); Dijkstra with a binary heap is O((V + E) log V). Space is O(V) for visited/parent data.
How do you keep comparisons fair?	Every algorithm runs on an identical copy of the maze with the same neighbour order, costs and definitions.
Why separate .h and .cpp?	The header is the interface other files depend on; the .cpp hides the implementation and keeps compile units independent.
Why separate the GUI from the solver?	The solver can be tested in the console, reused, and its results shown by any renderer without algorithm code in the UI.

14. Risks and mitigations

Risk	Mitigation
Building every feature at once	Follow Must, then Should, then Stretch, in phase order
GUI complexity blocks DSA progress	Prove algorithms in console tests before SFML
Animation freezes the app	Event-driven playback in the main loop; no long sleeps
Incorrect comparison statistics	Use real results, the same snapshot, consistent definitions, repeated timing
Weighted A* returns a non-optimal path	Use an admissible heuristic scaled by minimum step cost; test against Dijkstra
SFML build/linker problems	Match SFML binaries to compiler version and architecture; test a tiny window early
Scope creep	Treat Prim/Kruskal, challenge mode, themes and race mode as stretch

15. Final acceptance checklist

- Builds from documented instructions on the target Windows setup.
- Maze editing never allows invalid start/end placement or out-of-bounds access.
- DFS and BFS return legal paths when one exists and cleanly report no-path cases.
- BFS shortest-path behaviour validated on unweighted tests.

- Dijkstra and A* handle weighted terrain and agree with known results.
- Animation controls work and never change solver correctness.
- Live data-structure panels show actual algorithm state.
- Metrics are measured, consistently defined and not hard-coded.
- Every algorithm in the comparison uses the same maze and settings.
- README has build instructions, features, screenshots, algorithms and known limitations.
- Git history shows meaningful milestones; binaries excluded.
- A prepared demo maze and viva explanation are ready.

Next step: V0.2 Core DSA. Implement DFS and BFS with visited tracking, parent-based path reconstruction and no-path handling, and test both on hand-made mazes.